Hamiltonian-blind in-context propagator for quantum many-body dynamics

Architecture specification v0.1 (implementation hand-off)

October 2026

Abstract

This document specifies, to implementation level, a classical sequence model that forecasts the time evolution of local observables of an n-qubit system whose Hamiltonian is *unknown* and whose interaction graph is *unknown*, from a short window of past observables. The model is a space–time transformer over site tokens and pair features, with an *in-context structured generator* head that propagates a per-site latent state with a learned linear, stable generator. It is trained once on trajectories from a family of Hamiltonians and used zero-shot on unseen ones. The document covers: data generation, the full architecture with parameter values, the training program, evaluation metrics, baselines, and one reference benchmark. Everything should be implemented in PyTorch; shapes are given explicitly.

Contents

1 Problem statement and notation **2**

2 Data generation **3**
2.1 Hamiltonian family for the reference benchmark . . . 3
2.2 Initial states . . . 3
2.3 Exact time evolution . . . 3
2.4 Extracting the observables . . . 3
2.5 Optional shot noise . . . 3
2.6 Dataset layout and sizes . . . 4
2.7 Augmentation . . . 4

3 Architecture **4**
3.1 Overview . . . 4
3.2 Embeddings . . . 4
3.3 Spatial layer (geometry) . . . 5
3.4 Temporal layer (dynamics) . . . 5
3.5 Block structure and widths . . . 5
3.6 Head A: in-context structured generator (default) . . . 6
3.7 Head B: free multi-horizon readout (ablation) . . . 7
3.8 Probe head (diagnostic, training-time only) . . . 7
3.9 Parameter values . . . 7
3.10 Recap diagram . . . 7

4 Loss **7**

5 Training program **9**
5.1 Per-example sampling 9
5.2 Optimiser and schedule 9
5.3 Stages 9
5.4 Compute estimates (HPC config, $n = 10$, $T = 128$) 9

6 Metrics **10**

7 Reference benchmark **10**
7.1 Baselines 10
7.2 Expected outcome and what would falsify the design 11

8 Implementation plan for the student **11**

1 Problem statement and notation

Physical setting. We have n qubits (sites) evolving under an unknown, time-independent Hamiltonian H on an unknown interaction graph. Time is discretised with a known step Δt; the state at step t is $|\psi_t\rangle = e^{-iH\Delta t}\,|\psi_{t-1}\rangle$ (we set $\hbar = 1$). We never observe $|\psi_t\rangle$: we observe only the expectation values of all Pauli strings of weight ≤ 2.

Observables. With σ_i^a, $a \in \{x, y, z\}$, the Pauli operators on site i, define at every step t

$$\mathsf{N}_t[i, a] = \langle \sigma_i^a \rangle_t \in [-1, 1], \qquad \mathsf{N}_t \in \mathbb{R}^{n\times 3} \quad \text{(node observables)}, \tag{1}$$

$$\mathsf{E}_t[i, j, a, b] = \langle \sigma_i^a \sigma_j^b \rangle_t \in [-1, 1],\ i \neq j, \qquad \mathsf{E}_t \in \mathbb{R}^{n\times n\times 9} \quad \text{(edge observables)}, \tag{2}$$

with $\mathsf{E}_t[i, i, \cdot, \cdot] = 0$ by convention and the symmetry $\mathsf{E}_t[j, i, b, a] = \mathsf{E}_t[i, j, a, b]$. We write $o_t = (\mathsf{N}_t, \mathsf{E}_t)$ for the pair and $M = 3n + 9\,n(n-1)/2$ for the number of independent real numbers in o_t (for $n = 10$: $M = 435$; for $n = 16$: $M = 1128$).

Task. Given a *context* $o_0, \ldots, o_T$ and Δt, predict $o_{T+1}, \ldots, o_{T+\mathcal{H}}$ for a horizon $\mathcal{H}$. The Hamiltonian, its parameters and its graph are not given; the model must infer whatever it needs from the context.

Why this is not Hamiltonian learning. Even with the exact H, the future observables are not determined by o_T alone: o_T (weight-$\leq$ 2 expectations) does not determine $|\psi_T\rangle$, and o_{T+1} depends on weight-3 correlators of $|\psi_T\rangle$ that are not in o_T (operator growth). Any forecaster therefore needs *memory*: the past window carries information about the hidden state. This is why the model keeps a learned per-site latent state, and why a Hamiltonian-learning routine alone cannot serve as a forecasting baseline without being handed the hidden state (Section 7).

Symbols used below.

n	number of qubits (varies between train and test)
T	context length (sampled per training example)
$\mathcal{H}$	prediction horizon (number of future steps supervised)
d	node (site-token) width
d_e	edge (pair-feature) width
n_h	number of attention heads, head width $d_h = d/n_h$
L	number of blocks (each block = one spatial layer + one temporal layer)
m	per-site latent dimension of the generator head
K	number of generator coefficients $\theta \in \mathbb{R}^K$

2 Data generation

The generator produces trajectories $(o_0, \ldots, o_{L_{\text{traj}}-1})$ together with metadata (the Hamiltonian coefficients and the graph) that is used only for the auxiliary probe head and for evaluation, never as a model input.

2.1 Hamiltonian family for the reference benchmark

Transverse-field Ising model on a random graph:

$$H = -\sum_{(i,j)\in\mathcal{E}} J_{ij}\, Z_i Z_j \;-\; \sum_{i=1}^{n} h_i\, X_i. \tag{3}$$

Sampling procedure per trajectory:

1. Graph: Erdős–Rényi $G(n,p)$ with $p = 3/(n-1)$ (mean degree 3); resample until connected.
2. Couplings $J_{ij} \sim \mathcal{U}[0.5, 1.5]$ for $(i,j) \in \mathcal{E}$; fields $h_i \sim \mathcal{U}[0.5, 1.5]$.
3. Time step: $\Delta t = 0.1$ (fixed for the benchmark; the model still receives Δt as an input so that later datasets can vary it in $[0.05, 0.3]$).
4. Trajectory length $L_{\text{traj}} = 160$ steps ($t_{\max} = 16$ in units of $1/J$).

Later families (same code path, different sampler): XXZ with random couplings, disordered Heisenberg, power-law long-range Ising (held out), and Lindblad versions of the above.

2.2 Initial states

Random product states: each qubit in an independent Haar-random single-qubit state, $|\psi_0\rangle = \bigotimes_i \left(\cos\frac{\vartheta_i}{2}\,|0\rangle + e^{i\varphi_i}\sin\frac{\vartheta_i}{2}\,|1\rangle\right)$ with $\cos\vartheta_i \sim \mathcal{U}[-1,1]$, $\varphi_i \sim \mathcal{U}[0, 2\pi)$. These span the Hilbert space (they form a frame), correspond to the standard quench setting, and have low initial entanglement so that early-time dynamics is informative. Optionally (later) add states from depth-2 random circuits.

2.3 Exact time evolution

Work in the 2^n-dimensional computational basis with the state as a complex vector $\psi \in \mathbb{C}^{2^n}$.

- $n \le 12$: build H as a sparse matrix, compute $U = e^{-iH\Delta t}$ once per trajectory as a dense matrix (`scipy.linalg.expm`; 4096^2 complex128 $\approx 256\,\text{MB}$ at $n = 12$), then $\psi_{t+1} = U\psi_t$.
- $n \in [13, 16]$: use `scipy.sparse.linalg.expm_multiply` (Krylov) per step, or diagonalise once if memory allows.
- Verify $|\langle\psi_t|\psi_t\rangle - 1| < 10^{-10}$ and energy conservation $|\langle H\rangle_t - \langle H\rangle_0| < 10^{-8}$ on every trajectory (unit test).

2.4 Extracting the observables

Reshape ψ_t to a tensor of shape $(2, \ldots, 2)$ (n indices). For every pair $i < j$, compute the two-qubit reduced density matrix ρ_{ij} by contracting all other indices (`numpy.einsum` / `tensordot`); then

$$\mathsf{E}_t[i,j,a,b] = \mathrm{Tr}\big(\rho_{ij}\,\sigma^a \otimes \sigma^b\big), \qquad \mathsf{N}_t[i,a] = \mathrm{Tr}\big(\rho_{ij}\,\sigma^a \otimes I\big) \text{ (any } j). \tag{4}$$

Cost $O(n^2 2^n)$ per step. Fill $\mathsf{E}_t[j,i,b,a]$ by symmetry and $\mathsf{E}_t[i,i,\cdot,\cdot] = 0$. Store as `float32`.

2.5 Optional shot noise

To emulate finite measurement statistics with N_{shots} repetitions per Pauli setting, replace each expectation o by $o + \epsilon$, $\epsilon \sim \mathcal{N}\big(0,\, (1-o^2)/N_{\text{shots}}\big)$, clipped to $[-1, 1]$. Use $N_{\text{shots}} \in \{\infty, 10^4, 10^3\}$ as separate dataset variants. Noise is applied to the *inputs*; targets stay noiseless.

2.6 Dataset layout and sizes

One HDF5 (or sharded `.npz`) file per split with arrays

name	shape	dtype
`node`	$(N_{\text{traj}}, L_{\text{traj}}, n, 3)$	float32
`edge`	$(N_{\text{traj}}, L_{\text{traj}}, n, n, 9)$	float32
`dt`	$(N_{\text{traj}},)$	float32
`adj`	(N_{traj}, n, n)	bool (metadata only)
`J`	(N_{traj}, n, n)	float32 (metadata only; 0 off-graph)
`h`	(N_{traj}, n)	float32 (metadata only)
`seed`	$(N_{\text{traj}},)$	int64

Reference benchmark ($n = 10$): 8 000 train / 500 validation / 500 test trajectories, each with its own graph, couplings and initial state (≈0.6 MB per trajectory, ≈5 GB total). Local development set ($n = 8$): 1 000 / 100 / 100. Size-transfer test sets: $n = 12$ and $n = 14$, 300 trajectories each, generated with the same sampler.

2.7 Augmentation

At load time apply a random permutation π of the sites to `node`, `edge` (both axes) and the metadata. The model is permutation-equivariant by construction, so this augmentation only guards against accidental symmetry breaking (e.g. from padding) and serves as a unit test: the loss must be invariant to π up to floating-point error.

3 Architecture

3.1 Overview

The network has four parts (Figure 1):

1. **Embeddings**: node observables → site tokens $h_{i,t} \in \mathbb{R}^d$; edge observables → pair features $e_{ij,t} \in \mathbb{R}^{d_e}$.
2. **Body**: L blocks, each a *spatial layer* (attention among the n sites at fixed t, biased and value-augmented by the pair features, followed by a pair-feature update) and a *temporal layer* (causal attention along t for each site; causal convolution along t for each pair).
3. **Generator head** (Head A, the default): from the body's output at a context position t, build (i) per-site latents $z_{i,t} \in \mathbb{R}^m$ and (ii) a stable linear generator $\mathcal{G}_t \in \mathbb{R}^{nm \times nm}$ assembled from a global coefficient vector θ_t and the pair features. Future latents are $Z_{t+k} = \exp(k\Delta t\, \mathcal{G}_t) Z_t$; decoders map latents back to node and edge observables.
4. **Auxiliary heads**: a free multi-horizon head (Head B, ablation) and a probe head that reads out the Hamiltonian parameters (training-time diagnostic only).

No absolute site index is ever embedded: the only way sites are distinguished is through their observables and their pair features. This makes the model equivariant under relabelling of sites and independent of n.

3.2 Embeddings

For every $t \in \{0, \dots, T\}$ and site i:

$$h^{(0)}_{i,t} = \mathrm{MLP}_{\mathrm{n}}\big([\,\mathsf{N}_t[i,:],\ \log \Delta t\,]\big) + \mathrm{PE}(t), \qquad \mathrm{MLP}_{\mathrm{n}} : \mathbb{R}^4 \to \mathbb{R}^d, \tag{5}$$

$$e^{(0)}_{ij,t} = \mathrm{MLP}_{\mathrm{e}}\big(\mathsf{E}_t[i,j,:]\big)\ \ (i \neq j), \qquad e^{(0)}_{ii,t} = 0, \qquad \mathrm{MLP}_{\mathrm{e}} : \mathbb{R}^9 \to \mathbb{R}^{d_e}. \tag{6}$$

Both MLPs have one hidden layer of width $4d$ (resp. $4d_e$) with GELU. $\mathrm{PE}(t)$ is the standard sinusoidal encoding of the *integer index t within the window* (so it is relative to the window start, not absolute time), with frequencies $\omega_k = 10000^{-2k/d}$.

3.3 Spatial layer (geometry)

Acts independently at each time t on the set $\{h_i\}_{i=1}^n$ and $\{e_{ij}\}$ (we drop the index t). Pre-LayerNorm throughout. For each head $\eta = 1, \dots, n_h$:

$$q_i^\eta = W_q^\eta \,\mathrm{LN}(h_i), \quad k_j^\eta = W_k^\eta \,\mathrm{LN}(h_j), \quad v_j^\eta = W_v^\eta \,\mathrm{LN}(h_j) \qquad \in \mathbb{R}^{d_h}, \tag{7}$$

$$b_{ij}^\eta = w_b^{\eta\top} \,\mathrm{LN}(e_{ij}) \qquad \in \mathbb{R} \quad \text{(pair bias)}, \tag{8}$$

$$\alpha_{ij}^\eta = \mathrm{softmax}_j\Big(\tfrac{q_i^\eta \cdot k_j^\eta}{\sqrt{d_h}} + b_{ij}^\eta\Big), \tag{9}$$

$$u_i^\eta = \sum_{j=1}^{n} \alpha_{ij}^\eta \left(v_j^\eta + W_{ev}^\eta \,\mathrm{LN}(e_{ij})\right) \qquad \in \mathbb{R}^{d_h} \quad \text{(pair-augmented values)}, \tag{10}$$

$$h_i \leftarrow h_i + W_o\,[u_i^1; \dots; u_i^{n_h}], \tag{11}$$

$$h_i \leftarrow h_i + \mathrm{MLP}_{\mathrm{ff}}(\mathrm{LN}(h_i)), \qquad \mathrm{MLP}_{\mathrm{ff}} : d \to 4d \to d. \tag{12}$$

The self term $j = i$ is included (with $e_{ii} = 0$). The pair bias b_{ij} lets a pair with strong correlations attend to each other even when their node vectors are similar; the value augmentation lets two-body information flow into the site token. Then the pair features are updated from the sites they connect:

$$e_{ij} \leftarrow e_{ij} + \mathrm{MLP}_{\mathrm{eu}}\Big(\big[\mathrm{LN}(e_{ij});\ (W_a \mathrm{LN}(h_i)) \odot (W_b \mathrm{LN}(h_j))\big]\Big), \tag{13}$$

with $W_a, W_b : \mathbb{R}^d \to \mathbb{R}^{d_e}$ and $\mathrm{MLP}_{\mathrm{eu}} : 2d_e \to 4d_e \to d_e$.

After the update, symmetrise: $e_{ij} \leftarrow \frac{1}{2}(e_{ij} + e_{ji})$ (keeps the pair representation undirected; the direction-dependent information is already in E's (a, b) ordering, which is fed symmetrically at embedding time via $\mathsf{E}[j, i, b, a] = \mathsf{E}[i, j, a, b]$).

Cost per time slice. $O(n^2 d)$ for attention, $O(n^2 d_e)$ for the edge update.

3.4 Temporal layer (dynamics)

Acts independently on each site i (and each pair ij) across the window $t = 0, \dots, T$:

$$h_{i,t} \leftarrow h_{i,t} + \mathrm{CausalMHA}_t\big(\mathrm{LN}(h_{i,0:T})\big)_t, \qquad n_h \text{ heads, mask } t' \le t, \tag{14}$$

$$h_{i,t} \leftarrow h_{i,t} + \mathrm{MLP}'_{\mathrm{ff}}(\mathrm{LN}(h_{i,t})), \tag{15}$$

$$e_{ij,t} \leftarrow e_{ij,t} + \mathrm{CausalDWConv}_t\big(\mathrm{LN}(e_{ij,0:T})\big)_t, \qquad \text{depthwise, kernel 4, left-padded.} \tag{16}$$

The temporal attention uses rotary position embeddings (RoPE) on q, k in addition to the additive PE of Section 3.2; either alone is acceptable, both together is the safer default. Causality is mandatory: it is what allows one forward pass to produce predictions from every context position (dense supervision), and it is what makes the model usable online.

Why the temporal layer has no explicit geometry. It never compares tokens of different sites, but the vectors it attends over have already been mixed across sites by the preceding spatial layer (the residual stream carries that information), and after L blocks every token has a space–time receptive field of L hops $\times$ L steps, i.e. a light cone. An optional pair-bias on the temporal attention is *not* used in v0.1.

Cost per site. $O(T^2 d)$ for attention; $O(T d_e)$ for the edge convolution.

3.5 Block structure and widths

One block = spatial layer $\to$ temporal layer. L blocks are stacked; a final LN is applied to h and e. The output is $h_{i,t}^{(L)} \in \mathbb{R}^d$, $e_{ij,t}^{(L)} \in \mathbb{R}^{d_e}$ for all i, j, t.

3.6 Head A: in-context structured generator (default)

Everything in this subsection is computed at a chosen *anchor* position t of the context (several anchors per trajectory are sampled during training, Section 5); causality guarantees that only $o_{0:t}$ is used.

Summary and coefficients.

$$s_t = \frac{1}{n}\sum_{i=1}^{n} h_{i,t}^{(L)} \in \mathbb{R}^d, \qquad \theta_t = W_\theta\, s_t \in \mathbb{R}^K. \tag{17}$$

θ_t is a global, size-independent description of the dynamics (rates, frequencies, dissipation).

Per-site latents.

$$z_{i,t} = W_z\, h_{i,t}^{(L)} \in \mathbb{R}^m, \qquad Z_t = [z_{1,t}; \ldots; z_{n,t}] \in \mathbb{R}^{nm}. \tag{18}$$

The latent is taken from the transformer state, not from o_t alone, because o_t does not determine the future (Section 1): $z_{i,t}$ must carry the memory extracted from the window.

Generator assembly (size-independent, provably stable). Let $\bar{e}_{ij} = e_{ij,t}^{(L)}$. Define learned bases $A_k, S_k \in \mathbb{R}^{m\times m}$ $(k = 1..K)$ and a linear map $W_A : \mathbb{R}^{d_e} \to \mathbb{R}^{m\times m}$, a gate $g_{ij} = \sigma(w_g^\top \bar{e}_{ij}) \in (0,1)$. Build the $n \times n$ block matrices

$$\mathcal{A}_{ii} = \sum_{k=1}^{K} \theta_{t,k} A_k, \qquad \mathcal{A}_{ij} = g_{ij}\,\mathrm{reshape}_{m\times m}\big(W_A \bar{e}_{ij}\big)\ (i \neq j), \qquad \mathcal{S} = \mathrm{blockdiag}\Big(\sum_{k=1}^{K} \theta_{t,k} S_k\Big), \tag{19}$$

and the generator

$$\boxed{\mathcal{G}_t = (\mathcal{A} - \mathcal{A}^\top) \;-\; \mathcal{S}\mathcal{S}^\top \in \mathbb{R}^{nm\times nm}.} \tag{20}$$

The first term is skew-symmetric (a rotation of the latent: norm-preserving, the shape of closed-system Heisenberg dynamics); the second is negative semidefinite (a contraction: the shape of dissipation). Hence every eigenvalue of $\mathcal{G}_t$ has non-positive real part and $\|\exp(k\Delta t \mathcal{G}_t)\|_2 \leq 1$ for all k: rollouts cannot blow up. The off-diagonal blocks $\mathcal{A}_{ij}$ are where the *inferred interaction graph* lives (the gate g_{ij} is reported as the model's edge estimate). For closed-system data, $\|\mathcal{S}\|$ should be driven towards zero by training; its size on open-system data is a learned dissipation gauge.

Propagation. With $P_t = \exp(\Delta t\, \mathcal{G}_t)$ computed once by `torch.linalg.matrix_exp`,

$$Z_{t+k} = P_t^k\, Z_t, \qquad k = 1, \ldots, \mathcal{H}. \tag{21}$$

(Compute P_t^k by repeated multiplication of the *vector*, $O(\mathcal{H}\, n^2 m^2)$, never by forming powers of the matrix.)

Decoders.

$$\hat{\mathsf{N}}_{t+k}[i,:] = \tanh\big(D_{\mathrm{n}}\, z_{i,t+k}\big), \qquad D_{\mathrm{n}} : \mathbb{R}^m \to \mathbb{R}^3, \tag{22}$$

$$\hat{\mathsf{E}}_{t+k}[i,j,c] = \tanh\big(z_{i,t+k}^\top B_c\, z_{j,t+k} + \beta_c\big),\ c = 1..9, \qquad B_c \in \mathbb{R}^{m\times m}, \tag{23}$$

with the symmetry $\hat{\mathsf{E}}[j,i,(b,a)] = \hat{\mathsf{E}}[i,j,(a,b)]$ enforced by averaging the two orderings after the bilinear map. The tanh enforces $|\hat{o}| \leq 1$ so no separate clamp penalty is needed. Note the decoder is linear in z for nodes and bilinear for pairs: propagation is linear in the latent, outputs are at most quadratic in it (Koopman-style lifting with quadratic observables).

3.7 Head B: free multi-horizon readout (ablation)

$$\hat{\mathsf{N}}_{t+1:t+\mathcal{H}}[i,:,:] = \tanh\big(\mathrm{MLP}_{\mathrm{B}}(h_{i,t}^{(L)})\big) \in \mathbb{R}^{\mathcal{H}\times 3}, \tag{24}$$

$$\hat{\mathsf{E}}_{t+1:t+\mathcal{H}}[i,j,:,:] = \tanh\big(\mathrm{MLP}'_{\mathrm{B}}\big([\bar{e}_{ij};\ (W'_a h_{i,t}) \odot (W'_b h_{j,t})]\big)\big) \in \mathbb{R}^{\mathcal{H}\times 9}. \tag{25}$$

Same body, same loss; selected by a config flag. Used only to answer "does the structured generator help?".

3.8 Probe head (diagnostic, training-time only)

$$\hat{h}_i = w_h^\top h_{i,T}^{(L)}, \qquad \hat{J}_{ij} = w_J^\top \bar{e}_{ij}, \qquad \hat{a}_{ij} = \sigma(w_{adj}^\top \bar{e}_{ij}), \tag{26}$$

trained against the metadata $(h_i, J_{ij}, \mathrm{adj}_{ij})$ with a stop-gradient into the body (so the probe cannot shape the representation; it only reads it). The quality of this read-out answers "does the model implicitly learn the Hamiltonian?" without ever supplying H at inference.

3.9 Parameter values

symbol	local (laptop / 1 GPU)	HPC (benchmark)	meaning
n (train)	8	10	qubits in training data
d	128	256	site-token width
d_e	32	64	pair-feature width
n_h	4	8	attention heads (both layers)
L	3	6	blocks
m	32	64	per-site latent dim (nm=640, n=10)
K	32	64	generator coefficients
T (context)	$\mathcal{U}\{8,\dots,64\}$	$\mathcal{U}\{16,\dots,128\}$	sampled per example
$\mathcal{H}$ (horizon)	16	32	supervised future steps
anchors per trajectory	2	4	positions where Head A is evaluated
batch size	8	32	trajectories per step
parameters	≈1.2M	≈9M	total (Head A)

Dropout 0.0 (data is noiseless/abundant); weight decay 0.01; all linear layers initialised with PyTorch defaults except A_k, S_k (scaled $\mathcal{N}(0, 0.02^2)$) and W_A (zero-initialised, so the model starts with uncoupled sites and must *learn* to couple them).

3.10 Recap diagram

4 Loss

Let $\mathcal{T}_{\mathrm{anc}}$ be the set of anchor positions (Section 5) and $\mathcal{H}$ the horizon. Writing $\|\cdot\|_{\mathrm{n}}^2$ for the mean square over the $3n$ node entries and $\|\cdot\|_{\mathrm{e}}^2$ for the mean square over the $9n(n-1)$ off-diagonal edge entries,

$$\mathcal{L}_{\mathrm{pred}} = \frac{1}{|\mathcal{T}_{\mathrm{anc}}|\,\mathcal{H}} \sum_{t\in\mathcal{T}_{\mathrm{anc}}} \sum_{k=1}^{\mathcal{H}} w_k \Big(\|\hat{\mathsf{N}}_{t+k} - \mathsf{N}_{t+k}\|_{\mathrm{n}}^2 + \|\hat{\mathsf{E}}_{t+k} - \mathsf{E}_{t+k}\|_{\mathrm{e}}^2 \Big), \qquad w_k = 1. \tag{27}$$

(Uniform w_k in v0.1; try $w_k \propto k$ later to emphasise the horizon.)

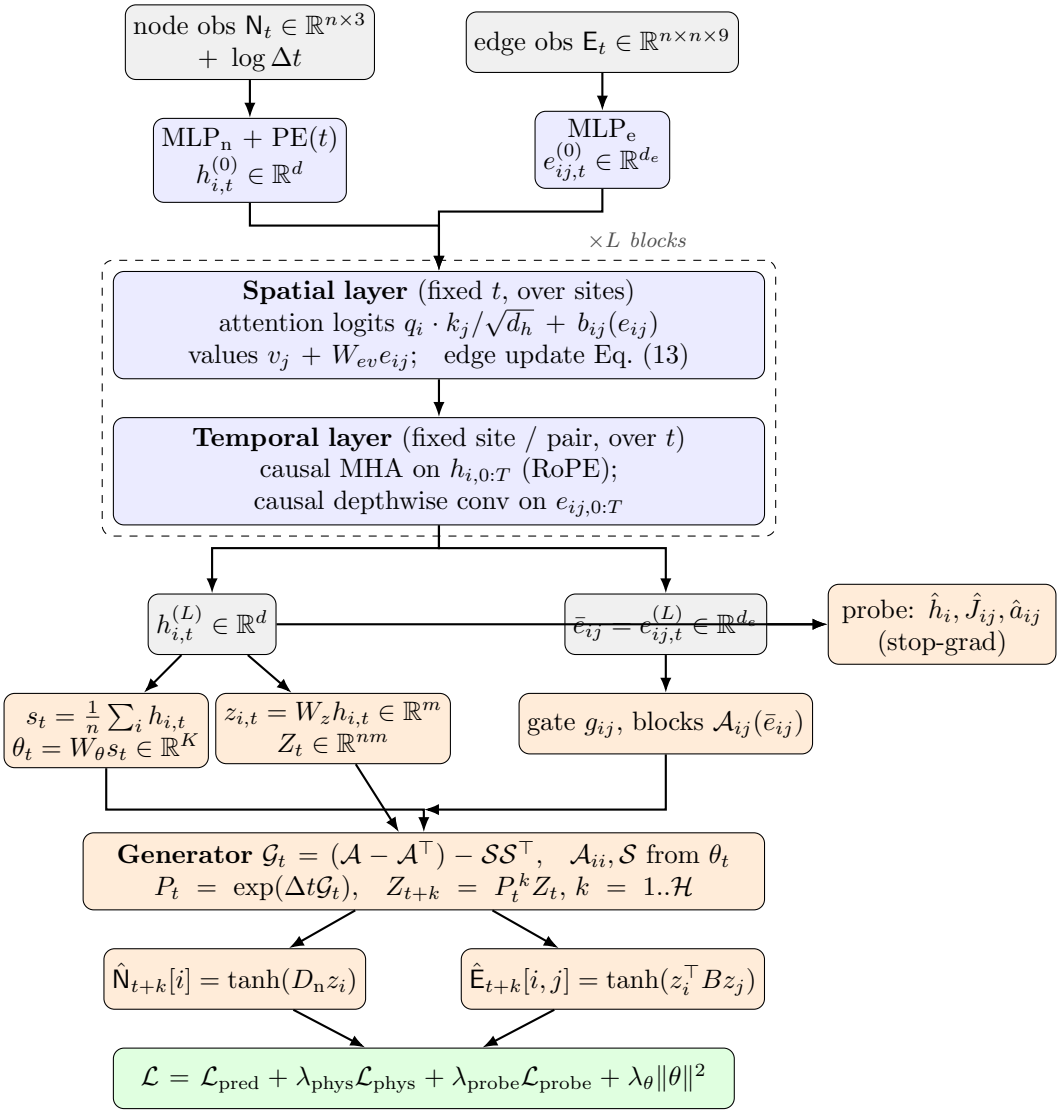


Figure 1: Data flow of the model (Head A). Grey: data; blue: body; orange: heads; green: loss. Nothing in the network depends on n or on a site index.

Physics penalty. For every predicted pair, reconstruct the two-qubit reduced density matrix

$$\hat{\rho}_{ij} = \tfrac{1}{4} \sum_{a,b \in \{0,x,y,z\}} c_{ab}\, \sigma^a \otimes \sigma^b, \quad c_{00} = 1,\ c_{a0} = \hat{\mathsf{N}}[i,a],\ c_{0b} = \hat{\mathsf{N}}[j,b],\ c_{ab} = \hat{\mathsf{E}}[i,j,a,b], \tag{28}$$

and penalise negativity: $\mathcal{L}_{\text{phys}} = \text{mean}_{i<j,\,k}\ \text{relu}\big(-\lambda_{\min}(\hat{\rho}_{ij})\big)$ (eigenvalues of a 4×4 Hermitian matrix; `torch.linalg.eigvalsh`). This is the cheapest way to force node and edge predictions to be mutually consistent with *some* quantum state.

Probe and regularisation. $\mathcal{L}_{\text{probe}} = \text{MSE}(\hat{h}, h) + \text{MSE}(\hat{J}, J) + \text{BCE}(\hat{a}, \text{adj})$ on stop-gradient features; $\mathcal{L}_\theta = \|\theta_t\|^2$.

Weights. $\lambda_{\text{phys}} = 0.1$, $\lambda_{\text{probe}} = 1$ (it cannot affect the body), $\lambda_\theta = 10^{-4}$.

5 Training program

5.1 Per-example sampling

For each trajectory in a batch: draw a context length T uniformly from the configured range; draw a window start t_0 uniformly so that $t_0 + T + \mathcal{H} < L_{\text{traj}}$; feed $o_{t_0:t_0+T}$; draw the anchor set $\mathcal{T}_{\text{anc}}$ as the last context position plus (anchors-1) positions uniform in $[t_0 + 8,\ t_0 + T)$. Supervise each anchor on the next $\mathcal{H}$ true steps. Because the body is causal, one forward pass of the body serves all anchors; only the head (one matrix exponential of size nm per anchor) is repeated.

5.2 Optimiser and schedule

AdamW, $\beta = (0.9, 0.95)$, peak learning rate 3×10^{-4}, linear warm-up over 2 000 steps, cosine decay to 3×10^{-5}; gradient-norm clipping at 1.0; bfloat16 autocast for the body, **float32 for the generator head** (the matrix exponential and the eigenvalue penalty are numerically sensitive).

5.3 Stages

1. **Sanity (local, $n = 8$).** 1 000 trajectories, local config, $\sim$20 k steps. Required checks before moving on: (i) overfit a single batch to $\mathcal{L}_{\text{pred}} < 10^{-4}$; (ii) loss invariant under site permutation; (iii) $\max \text{Re}\,\text{eig}(\mathcal{G}_t) \le 10^{-5}$ on random inputs; (iv) the forward pass runs unchanged for $n = 8$ and $n = 10$ data (no shape depends on n).
2. **Benchmark (HPC, $n = 10$).** 8 000 trajectories, HPC config, 150 k steps ($\approx$ 1 A100-day at batch 32). Evaluate every 5 k steps on the validation set with the metrics of Section 6.
3. **Transfer.** Without retraining, evaluate on the $n = 12, 14$ test sets and on the held-out family. Then (separate run) add families to the training mix and repeat.
4. **Noise and open systems** (later): add the N_{shots} variants and Lindblad data to the mix; monitor $\|\mathcal{S}\|$.

5.4 Compute estimates (HPC config, $n = 10$, $T = 128$)

Tokens per trajectory: $n(T{+}1) = 1290$ site tokens and $n^2(T{+}1) \approx 1.3 \times 10^5$ pair features of width 64. Body FLOPs per trajectory $\approx$ 1–2 GFLOP; head: 4 matrix exponentials of a 640×640 matrix. A batch of 32 is well under 100 ms on an A100. Memory is dominated by the pair features: $32 \times 1.3 \times 10^5 \times 64 \times 2$ bytes ≈ 0.5 GB per block of activations; use activation checkpointing per block if $L = 6$ exceeds memory.

6 Metrics

All metrics are computed on the test split, from the last context position, with the context length T swept over $\{8, 16, 32, 64, 128\}$.

1. **Horizon error.** $\mathrm{RMSE}_{\mathrm{n}}(k) = \sqrt{\|\hat{\mathsf{N}}_{T+k} - \mathsf{N}_{T+k}\|_{\mathrm{n}}^2}$ and $\mathrm{RMSE}_{\mathrm{e}}(k)$ likewise, averaged over test trajectories, for $k = 1..\mathcal{H}$ (and beyond $\mathcal{H}$ up to 64 to test extrapolation of the generator). Observables live in $[-1, 1]$, so these are absolute errors on a fixed scale.
2. **Predictability horizon.** $k^\star(T) = \max\{k : \ \mathrm{RMSE}_{\mathrm{n}}(k') \le \epsilon \ \forall k' \le k\}$ with $\epsilon = 0.05$. The curve $k^\star$ versus T is the paper's central object (the *context–horizon curve*).
3. **Graph recovery.** AUROC of the gate g_{ij} (and of the time-averaged spatial attention $\bar{\alpha}_{ij}$) against the true adjacency, on test trajectories. Measures whether the model discovers the interaction structure.
4. **Probe quality.** R^2 of $\hat{J}_{ij}$ on true edges and of $\hat{h}_i$; AUROC of $\hat{a}_{ij}$.
5. **Dissipation gauge.** $\|\mathcal{S}\|_F$ averaged over test anchors (should be ≈ 0 on closed-system data).
6. **Cost.** Wall-clock per predicted step at inference, for the model and for each baseline, on the same hardware.

7 Reference benchmark

Benchmark B1 (TFIM on random graphs, closed system, noiseless). Family of Section 2.1, $n = 10$, $\Delta t = 0.1$, 500 test trajectories with unseen graphs, couplings and initial states; report $\mathrm{RMSE}_{\mathrm{n,e}}(k)$ for $k \le 64$ at $T \in \{8, 16, 32, 64, 128\}$, and $k^\star(T)$.

7.1 Baselines

1. **Persistence.** $\hat{o}_{T+k} = o_T$. The "do nothing" floor; at $\Delta t = 0.1$ it is not trivial at small k.
2. **In-context linear map (DMD / DLinear).** Flatten $o_t \in \mathbb{R}^M$; fit $W \in \mathbb{R}^{M \times M}$ by ridge regression on the T context pairs (o_t, o_{t+1}) with $\lambda_{\mathrm{ridge}} = 10^{-3}$, then roll out $\hat{o}_{T+k} = W^k o_T$. Also the variant with a window of $w = 4$ past steps as input ($W \in \mathbb{R}^{M \times 4M}$). This is the baseline that a recent paper (arXiv:2610.10697) found competitive with transformers on a single fixed system; the claim to establish is that the learned prior wins at small T and across systems.
3. **Oracle.** Exact propagation with the true H *and the true hidden state* $|\psi_T\rangle$. Upper bound for everything.
4. **Hamiltonian learning + oracle state.** Estimate (J_{ij}, h_i) from the context observables only, then propagate exactly from the true $|\psi_T\rangle$. The estimator uses the single-site Heisenberg equations of Eq. (3), which close on weight-≤ 2 data:

$$\tfrac{d}{dt}\langle X_i\rangle = 2\sum_j J_{ij}\langle Y_i Z_j\rangle, \qquad \tfrac{d}{dt}\langle Y_i\rangle = -2\sum_j J_{ij}\langle X_i Z_j\rangle + 2h_i\langle Z_i\rangle, \qquad \tfrac{d}{dt}\langle Z_i\rangle = -2h_i\langle Y_i\rangle, \tag{29}$$

   with the left-hand sides from centred finite differences $(o_{t+1} - o_{t-1})/2\Delta t$. This is a linear least-squares problem in the $n + n(n-1)/2$ unknowns (all pairs allowed, i.e. structure unknown; add an ℓ_1 penalty 10^{-3} to encourage sparsity), with $3n(T-1)$ equations. Its gap to the oracle isolates *identification error*; its gap to the model isolates the *value of the hidden state*. *Note: there is no observable-only HL forecaster: without $|\psi_T\rangle$, even the exact H cannot be propagated. This is the formal reason the task differs from Hamiltonian learning, and it should be stated in the paper.*

7.2 Expected outcome and what would falsify the design

Expected ordering at $T = 16$: oracle < HL+oracle ≈ model < DMD < persistence in error; at $T = 128$: DMD closes much of its gap on a single system, the model should remain at least as good and its $k^\star(T)$ should grow faster at small T. The design is falsified if the model does not beat the ridge DMD baseline at $T \leq 32$, or if $k^\star$ does not improve with T (which would mean the context is not being used for identification). Graph-recovery AUROC above 0.9 is the expected signature that the spatial attention has found the structure.

8 Implementation plan for the student

Suggested repository layout and order of work. Each step ends with a test that must pass before the next.

1. `data/generate.py`: sampler for Section 2; CLI arguments `--n --ntraj --L_traj --dt --family --seed --out`. Test: norm and energy conservation; `E` symmetry; values in $[-1, 1]$; regenerate one trajectory from its seed bit-for-bit.
2. `data/dataset.py`: windowed loader with context/anchor sampling and permutation augmentation. Test: a batch with π applied gives permuted outputs.
3. `model/embed.py`, `model/spatial.py`, `model/temporal.py`, `model/body.py`. Test: output equivariance under π to 10^{-5}; identical code path for two values of n.
4. `model/head_generator.py`: Eq. (20), matrix exponential, decoders. Test: $\max \mathrm{Re}\,\mathrm{eig}(\mathcal{G}) \leq 10^{-5}$; $\|P^k Z\| \leq \|Z\|$.
5. `model/head_free.py`, `model/probe.py`.
6. `train.py`: loss of Section 4, schedule of Section 5, checkpointing, W&B/TensorBoard logging of all metrics. Test: single-batch overfit.
7. `eval/metrics.py`, `eval/baselines.py` (persistence, ridge DMD, HL least squares, oracle using stored seeds to regenerate $|\psi_T\rangle$). Test: oracle error is 0 to float precision.
8. Run Stage 1 locally; then Stage 2 on HPC (one GPU; the code must not require multi-GPU). Produce the context–horizon figure and the graph-recovery figure.

Keep every hyperparameter in a single YAML config with the two presets of Section 3.9; log the git commit and the config with every checkpoint.